SCALER · SESSION 2

Computer Networks

Seven syscalls make a server. Seven layers of nesting carry eleven bytes.

Today we open both ends.

OSI · SS7 · BBR · TCP · UDP · QUIC · SMTP · POP · FTP



AGENDA

Five stops, one idea running through all of them

One layer’s headers are another layer’s body — in Ethernet, in SS7, in email, in FTP.

01

Sockets

Six syscalls and you have a server. The seventh is where the bugs live.

02

OSI

Seven layers — over copper, over radio, and over a satellite 550 km up.

03

SS7

A second internet, older than TCP/IP, and why an SMS is 140 bytes.

04

TCP · UDP · QUIC

Handshake, teardown, SYN cookies, and the round-trip tax.

05

Text protocols

SMTP, POP and FTP — typed by hand into telnet.

01

Sockets

Everything above this is a convention. Everything below is somebody else's problem.

A TCP server is seven system calls

echo_server.c

```c
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>

int main() {
    int server_fd = socket(AF_INET, SOCK_STREAM, 0);
    struct sockaddr_in addr = {0};
    addr.sin_family = AF_INET;
    addr.sin_addr.s_addr = INADDR_ANY;
    addr.sin_port = htons(2026);
    bind(server_fd, (struct sockaddr*)&addr, sizeof(addr));
    listen(server_fd, 1);

    while (1) {
        int client_fd = accept(server_fd, NULL, NULL);
        char buf[4096];
        int n = read(client_fd, buf, sizeof(buf));
        write(client_fd, buf, n);
        close(client_fd);
    }
}
```

THE SIX

socket → bind → listen → accept → read → write → close

Everything else in this file is address bookkeeping.

LISTEN(FD, 1)

The 1 is the accept-queue depth, not the connection limit. Remember it — it comes back when we talk about SYN floods.

HTONS

Host to network short. The wire is big-endian; your laptop probably isn't.

The client is the same calls, mirrored

client.c

```c
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>

int main() {
    int fd = socket(AF_INET, SOCK_STREAM, 0);
    struct sockaddr_in addr = {0};
    addr.sin_family = AF_INET;
    addr.sin_addr.s_addr = inet_addr("127.0.0.1");
    addr.sin_port = htons(8080);
    connect(fd, (struct sockaddr*)&addr, sizeof(addr));

    write(fd, "hello", 5);

    struct linger l = {1, 0}; // abortive close: sends RST instead of FIN
    setsockopt(fd, SOL_SOCKET, SO_LINGER, &l, sizeof(l));
    close(fd);

    return 0;
}
```

NO BIND, NO LISTEN

The kernel picks an ephemeral source port for you. connect() is what triggers the handshake.

SO_LINGER {1, 0}

A normal close() sends FIN — a polite “I’m done sending”, and the socket sits in TIME_WAIT for 2 MSL.

Linger with a zero timeout sends RST instead: the connection is torn down immediately, unsent data is discarded, and there is no TIME_WAIT.

Fast. Also lossy. Use it to demonstrate the difference, not in production.

02

OSI

A 1984 committee model that has outlived almost everything built on top of it.

OSI · L1–L7

Seven layers, each one refusing to care about the others

L7	APPLICATION	HTTP · FTP · DNS · SMTP
L6	PRESENTATION	Encryption, compression, encoding
L5	SESSION	Session management, authentication
L4	TRANSPORT	TCP · UDP
L3	NETWORK	IP · ICMP · routing
L2	DATA LINK	Ethernet · MAC · ARP · switches
L1	PHYSICAL	Cables, signals, hardware

TCP · AT L4

- ✔ Connection-oriented
- ✔ Reliable, ordered delivery
- ✔ Flow and congestion control
- ✔ Error recovery
- ⏱️ Higher latency
- 📦 HTTP, FTP, SMTP, SSH

UDP · ALSO AT L4

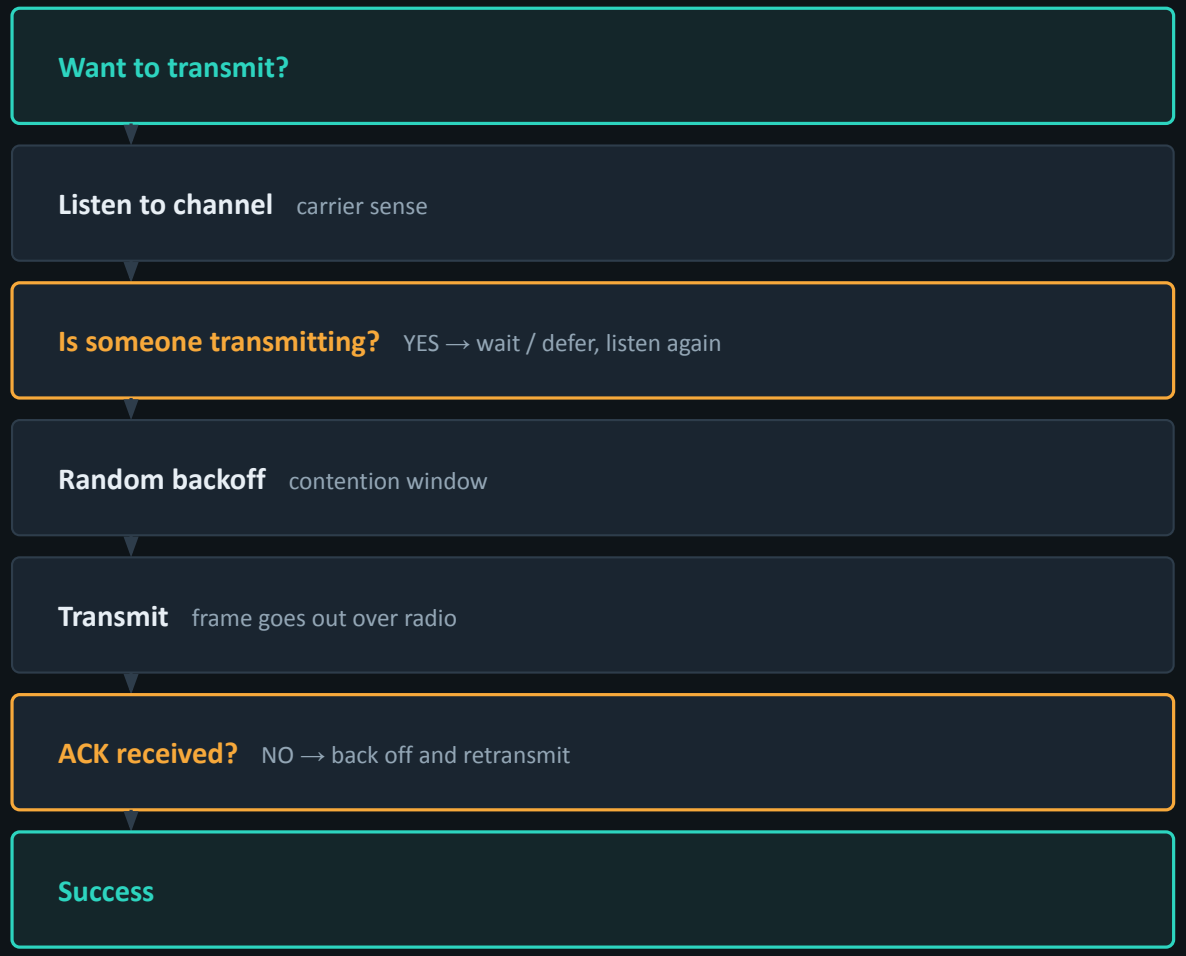
- ✔ Connectionless
- ✔ Fast, low overhead
- ✘ No delivery guarantee
- ✘ No ordering
- ⏱️ Low latency
- 📦 DNS, VoIP, video calls, gaming

OSI · L1 / L2

On a wire you detect collisions. On radio you can only avoid them.

Same L3 and above. Completely different L1 and L2 problem.

CSMA/CA — 802.11



WHY NOT CSMA/CD

A radio cannot listen while it transmits — its own signal drowns everything else out. So Wi-Fi cannot detect a collision the way Ethernet can. It avoids instead: sense, back off, transmit, and demand an explicit ACK for every single frame.

SO WHERE DOES THE TIME GO

≈2 ms of contention delay before a frame even leaves.

Every frame is individually acknowledged at L2 — long before TCP at L4 has any idea anything happened.

OSI · L1

Your packet can go to space and back, and nothing above L2 notices

Starlink is a physical layer. That is the entire point of having layers.



THE WHOLE ARGUMENT FOR LAYERING

Nine physical hops, one of them 550 km straight up, and the IP header written by your laptop arrives unchanged — except TTL, checksum and any NAT rewrite. L3 and above never learn that space was involved.

03

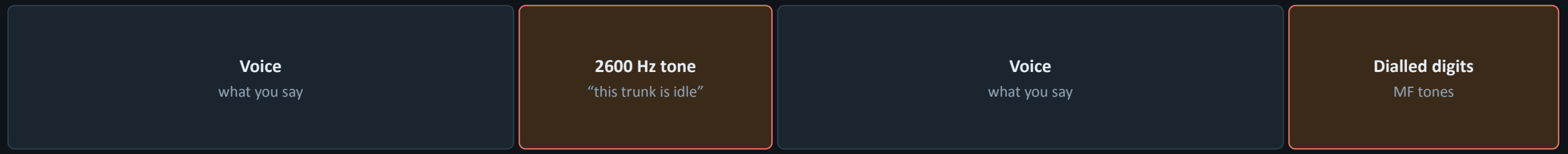
SS7

There is a second internet. You used it this morning and you have never seen a packet from it.

SS7 · 1975

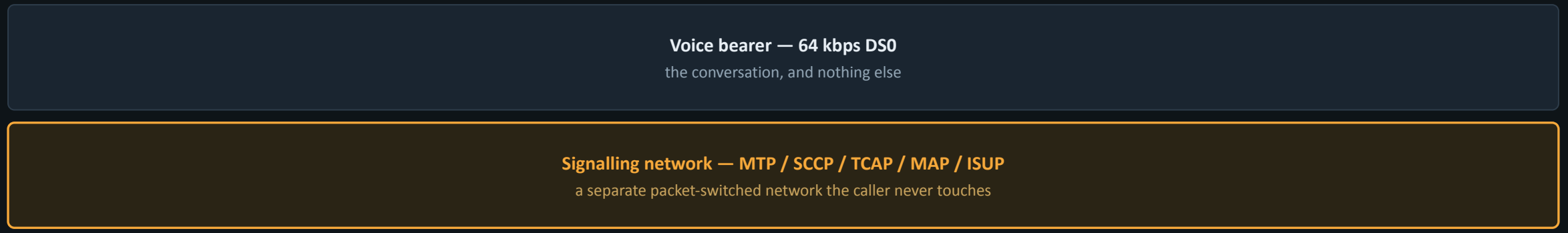
SS7’s one big idea: get the signalling out of the voice channel

BEFORE · in-band signalling



One channel carries both the conversation and the control. Anyone who can make the sound can make the decision.

AFTER · out-of-band signalling (SS7, 1975)



Same separation of concerns as a control plane and a data plane. Fifty years old, and every network you build still does it.

SS7 VS TCP/IP

Same seven jobs, different names, very different trust model

OSI	SS7 STACK	TCP/IP STACK
L7	**MAP · INAP · CAP · ISUP** Call setup, roaming, SMS	**HTTP · DNS · SMTP · SIP** Application protocols
L5–L6	**TCAP** Dialogues, components, ASN.1 BER	**TLS / app framing** No distinct layer — folded into the app
L4	**SCCP (classes 0–3)** Subsystem numbers, Global Title Translation	**TCP / UDP** Ports, byte-stream sequencing, windowing
L3	**MTP3** Point-code routing, SLS load sharing	**IP** Best-effort datagrams, fragmentation, TTL
L2	**MTP2** Signal units, FSN/BSN, CRC-16, retransmission	**Ethernet · PPP · HDLC** Framing, MAC addressing, FCS
L1	**MTP1** 64 kbps DS0 on E1 / T1	**Copper · fibre · radio** Any bit-carrying medium

SS7 is a whole stack; TCP is one protocol at L4. The fair comparison is stack to stack.

PACKET SHAPE

One layer’s headers are another layer’s body

A browser on 192.168.1.42 sending GET / to example.com. Read top to bottom = peeling the onion at the receiver.

L2 — ETHERNET II FRAME

Dst MAC	Src MAC	EtherType	Payload — the entire IP packet	FCS
6 B	6 B	2 B · 0x0800	46–1500 B (MTU)	4 B · CRC-32

L3 — IPv4 PACKET (20 B header)

Ver/IHL	Total Len	TTL	Proto	Checksum	Src IP	Dst IP	Payload — the entire TCP segment
4b+4b	2 B	1 B	6 = TCP	2 B	4 B	4 B	

L4 — TCP SEGMENT (20 B header)

Src Port	Dst Port	Seq	Ack	Flags	Window	Payload — application data
2 B	2 B	4 B	4 B	SYN/ACK/FIN	2 B	≤ MSS, typically 1460 B

L7 — APPLICATION DATA

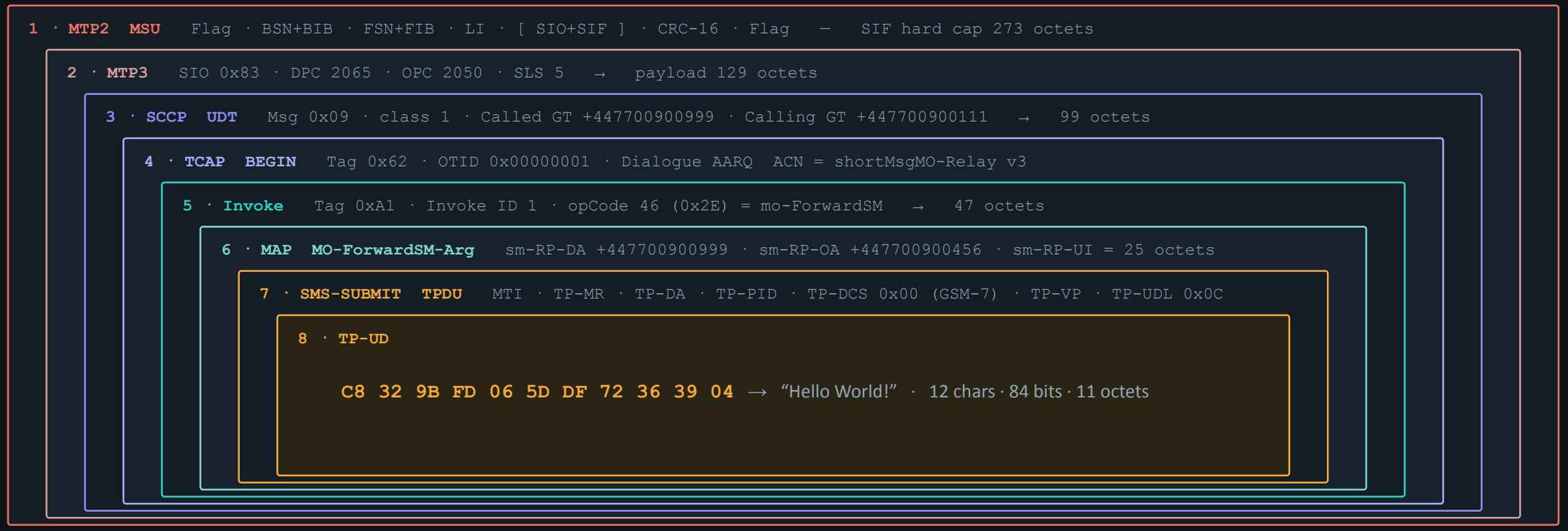
GET / HTTP/1.1 · Host: example.com — an opaque byte stream TCP neither knows nor cares about

HTTP request · TLS record · SMTP command · SIP INVITE · anything

Byte budget: 1500 B MTU − 20 B IP − 20 B TCP = 1460 B MSS. Add 14 B Ethernet + 4 B FCS and the wire frame is 1518 B.

Seven layers of nesting to carry eleven bytes

A GSM handset sending “Hello World!” — MO-ForwardSM, all the way down to the octets.



Identical picture to Ethernet ⊃ IP ⊃ TCP ⊃ HTTP. Different names, same onion. The payload just caps at 272 octets instead of 1460 B.

SS7 · MAP

An SMS is 140 bytes because a 1980s signalling packet caps at 272

Three independent length limits, stacked. The smallest one wins, and it defines the product.

272 octets

SIF — MTP2's hard ceiling

The physical limit of a single Message Signal Unit.

~200 octets

sm-RP-UI — MAP's field

What MAP is willing to carry as opaque user data.

140 octets

TP-UD — the SMS itself

Chosen so the whole MAP operation always fits in one MSU.

NO SEGMENTATION, NO REASSEMBLY

The 140-octet limit exists so that one text message is always exactly one signalling packet. Nothing to fragment, nothing to reorder, nothing to hold state for — which is precisely why SMS ran on 1980s hardware, and why it still works when the data network is down.

SS7 · GSM-7

160 characters in 140 bytes — because ASCII wastes a bit

“I said 140 bytes, but you can send 160.” The discrepancy is a packing trick from mobile engineers.

140
octets in TP-UD
the hard field size

× 8
bits per octet
= 1120 bits available

÷ 7
bits per character
GSM 7-bit alphabet

160
characters
TP-DCS = 0x00

THE PACKING TRICK

Each character is 7 bits, not 8. The septets are pushed into one continuous bit stream and re-sliced into octets — so after the first character, character boundaries stop lining up with byte boundaries.

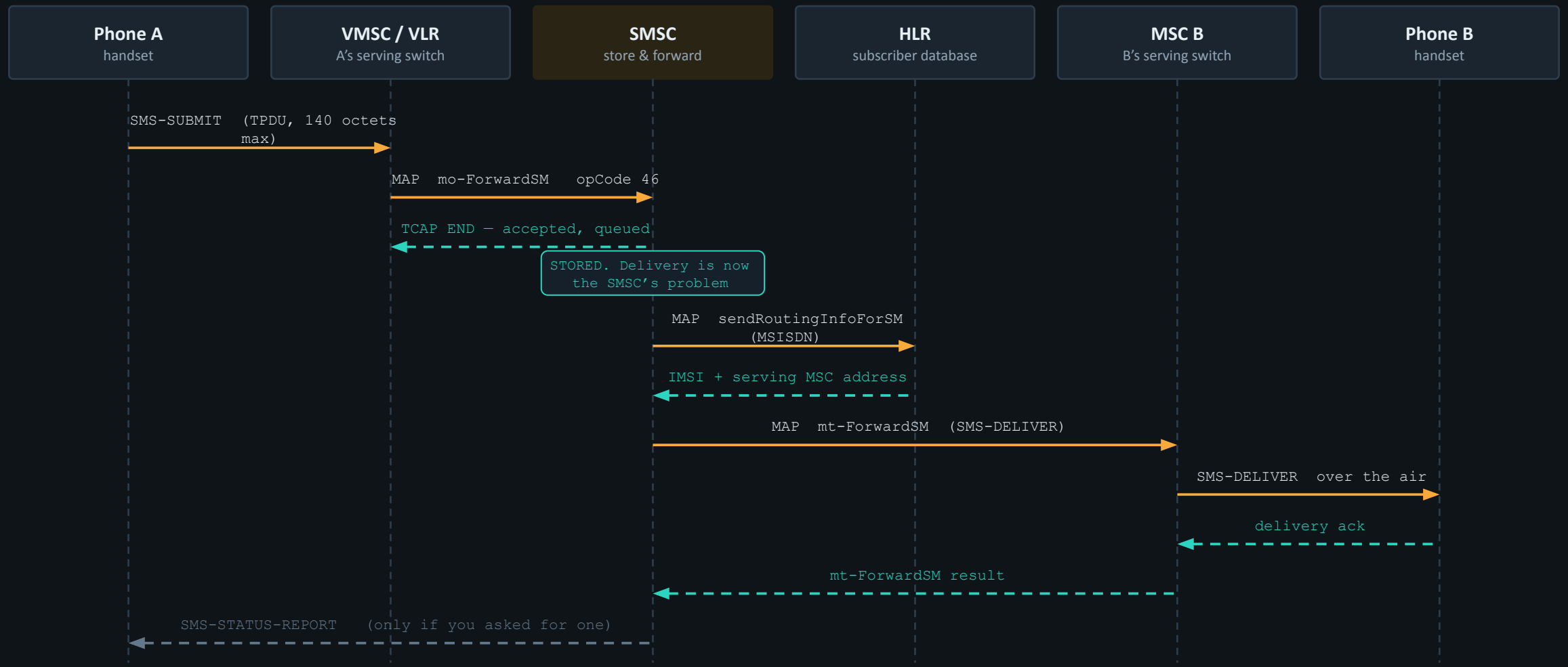
12 chars × 7 bits = 84 bits ⟶ 11 octets (88 bits), with 4 bits of zero padding.

AND THE MOMENT IT BREAKS

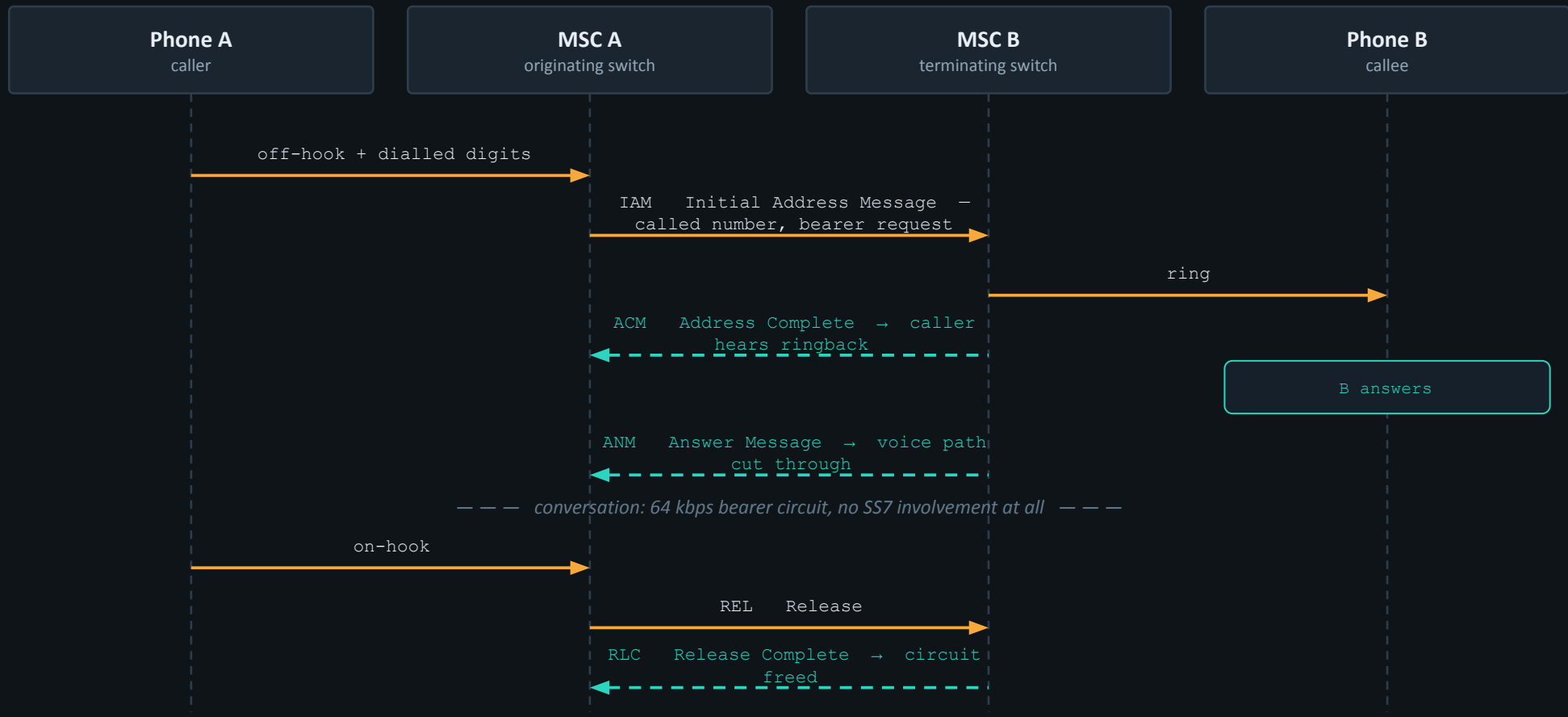
Switch TP-DCS to UCS-2 for emoji or any non-Latin script and it is 140 ÷ 2 = 70 characters. Same 140-octet field either way.

That is why one emoji can more than halve your message length — and why it silently costs you a second SMS.

Sending a text is a database lookup wrapped in a queue

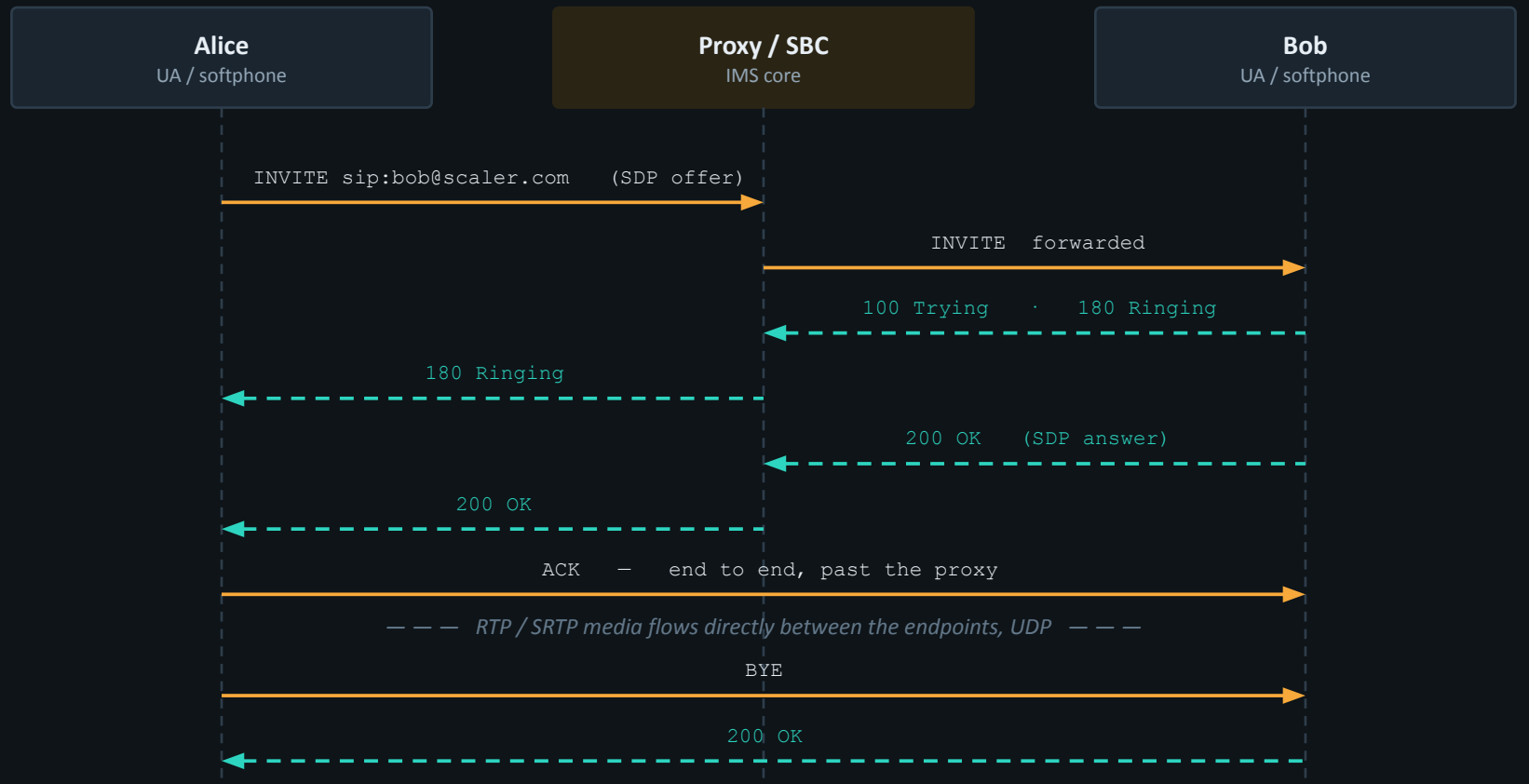


A phone call is signalling on one network, voice on another



Four messages set up a call. During the conversation itself the signalling network is completely idle — it did its job at the edges.

In 5G the same call is HTTP-shaped text over IP



100, 180, 200 — SIP borrowed HTTP's status codes wholesale. Signalling is text; media is RTP over UDP. Control and data stay on separate paths.

Try it yourself: Asterisk or FreeSWITCH, a softphone (Linphone, Zoiper), and Wireshark with the SIP dissector on.

SS7 VS TCP

SS7 makes every hop reliable. TCP makes only the endpoints reliable.

FUNCTION	SS7	TCP
Sequencing	FSN — 7 bits, per signal unit, wraps at 128	Sequence number — 32 bits, per byte
Acknowledgement	BSN + BIB, piggybacked on every SU	ACK number + flag, cumulative (+ SACK)
Retransmission	BIB toggle; PCR on long / satellite links	RTO timers, fast retransmit on 3 dup-ACKs
Flow control	SIB in an LSSU — a blunt stop/go signal	Sliding window advertised in every segment
Error detection	CRC-16 per signal unit	16-bit ones-complement checksum
Addressing	Point Code (OPC/DPC) + SSN + Global Title	IP address + port number
Connection setup	Connectionless; SCCP class 2/3 if needed	3-way handshake (SYN, SYN-ACK, ACK)
Congestion control	Hop-by-hop via MTP3 transfer-controlled	End-to-end (Reno, CUBIC, BBR)
Scope of reliability	Per hop, between adjacent signalling points	End to end, across arbitrary hops
Max payload	272 octets (SIF)	~1460 B typical, 64 KB with window scaling
Security	None — trust by point code	None natively; TLS layered above

Both stacks solve the same eleven problems. The only real disagreement is which layer gets to be responsible.

CONGESTION CONTROL · 2016

BBR stops treating packet loss as the signal for congestion

Loss-based TCP fills the buffer until something drops. On a fat, long, or lossy link that is exactly wrong.



THE SAWTOOTH IS THE BUG

Reno and CUBIC grow until a packet is dropped, then halve. So they need loss to make progress — and on a link that drops packets for reasons other than congestion, they crawl.

WHAT BBR MEASURES INSTEAD

Bottleneck bandwidth × round-trip propagation time. Send at the rate the path can actually carry, keeping the queue near empty. Google reports up to 2,000× throughput on some benchmarks — and much lower latency.

youtu.be/VIX45zMMZG8 · cloud.google.com/blog — “TCP BBR comes to GCP” · bufferbloat, 2011: youtu.be/qbIozKVz73g

04

TCP · UDP · QUIC

No class with “computer networks” in the title is complete without the handshake.

Three messages, because each direction needs its own sequence number confirmed

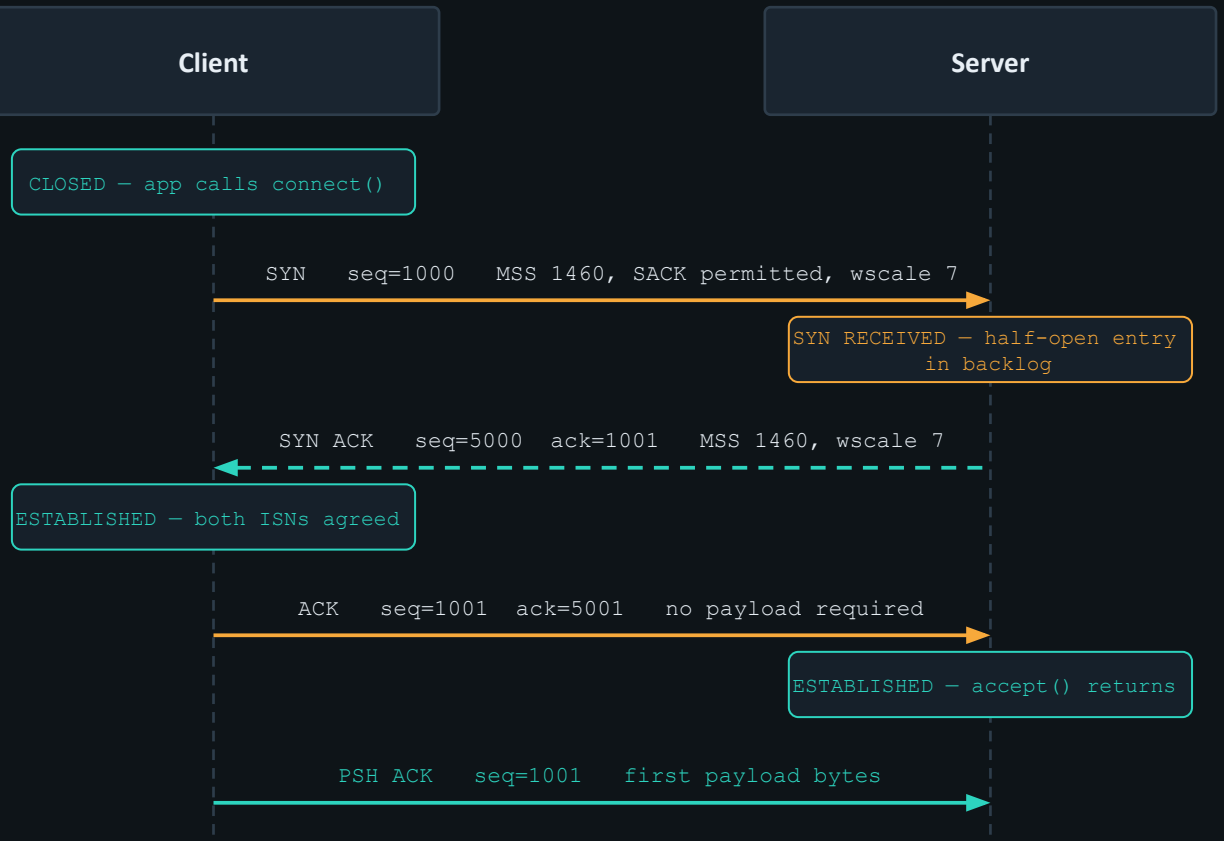
WHY 3

Segment 1 announces the client's ISN.

Segment 2 acknowledges it and announces the server's.

Segment 3 acknowledges the server's.

The server's SYN is what forces the third leg.



RANDOM ISN

Early stacks counted predictably. Guess the next ISN and you can inject forged segments blind — the Mitnick attack.

RFC 6528: keyed hash of the 4-tuple + timer.

SYN and FIN each consume a sequence number. That is why ack=1001 answers seq=1000 even though the SYN carried no payload — otherwise a bare SYN would be indistinguishable from a retransmission.

TCP · DEFENCE

SYN cookies: keep no state, and there is nothing to exhaust

Between segment 1 and segment 3, the server is holding an entry in a finite backlog. That backlog is the target.

THE ATTACK

Send SYNs from spoofed source addresses and never complete the handshake.

Each one costs the attacker a single packet and costs the server a half-open entry that lingers until it times out. Fill the backlog and legitimate connections are refused.

THE DEFENCE · 1996

Do not store anything. Encode the connection parameters into the server's own initial sequence number — a keyed hash of the 4-tuple, a coarse timestamp and the MSS.

The client's ACK carries that number back, incremented. The server recomputes, validates, and reconstructs the entire connection from a packet it has never seen before.

THE COST

Most TCP options do not survive — you have 32 bits of ISN to work with, so window scale, SACK and timestamps are mostly lost for that connection. Which is why cookies switch on under pressure rather than always.

HOMEWORK · TWO QUESTIONS

1. TCP SYN cookies are 1996. HTTP cookies are… which came first, and did either name influence the other?

2. Where else have you used this exact trick — push the state into a token the client hands back?

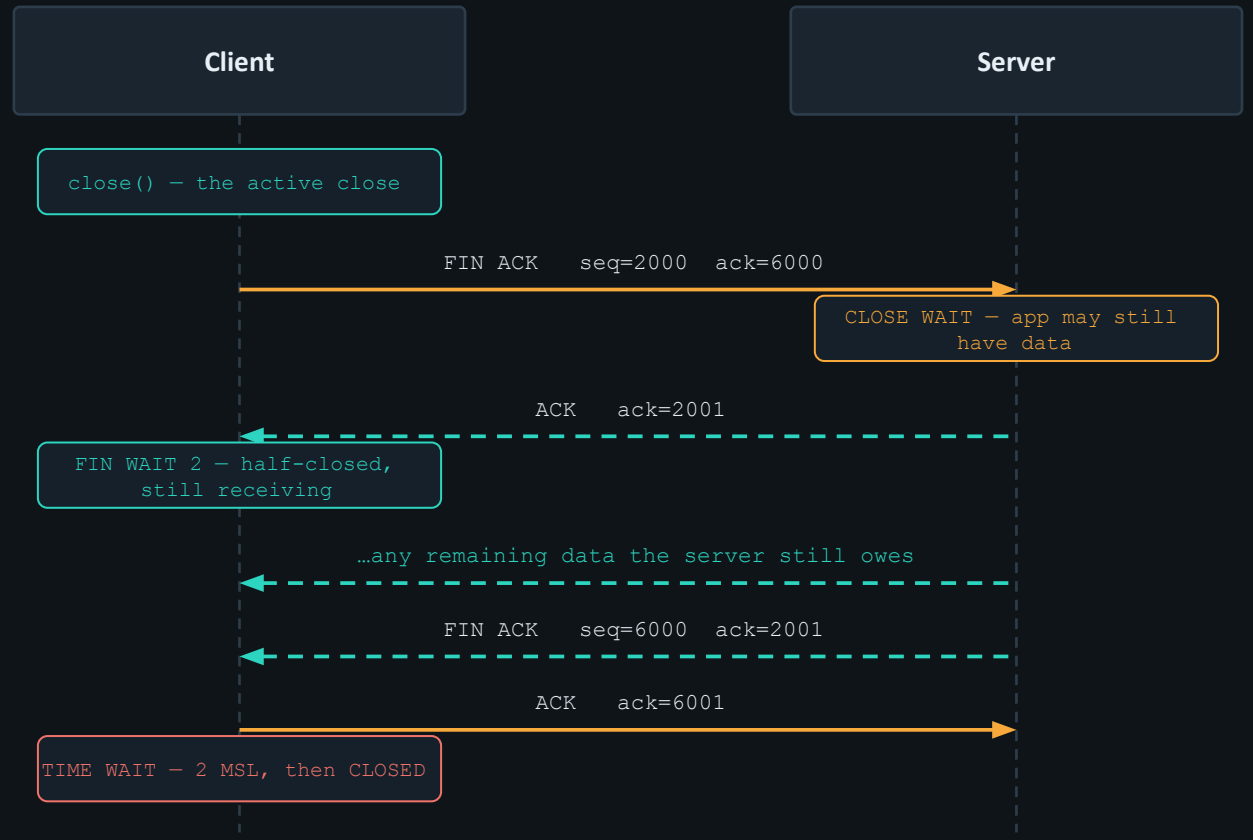
Closing takes four segments, not three

HALF-CLOSE

Not an error — a real state.

After the first FIN and its ACK the connection is one-directional. The client sends no more; the server keeps sending until it closes its own side.

SYN+ACK can be combined. FIN+ACK cannot.



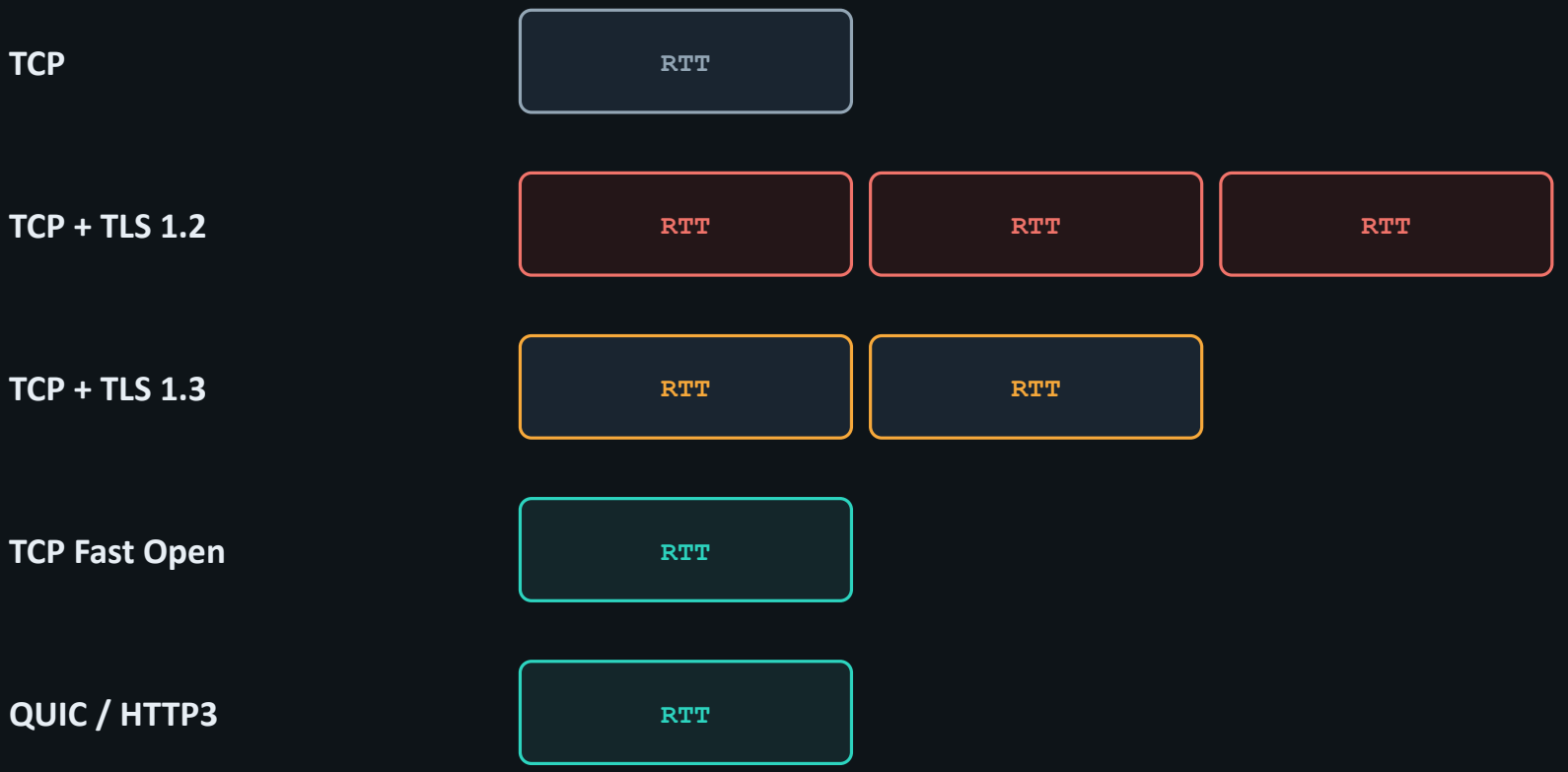
SOCKETS STUCK IN CLOSE_WAIT

Almost always an application that forgot to call close().

Not a network problem. Look at your code, not your switch.

TIME_WAIT is why you get “address already in use” on server restart — and why SO_REUSEADDR exists.

Every round trip before your first byte is a tax, and everything modern is an attack on it



Bandwidth you can buy. Round trips you cannot — Mumbai to Virginia is ~190 ms and no amount of money changes the speed of light.

UDP

UDP is TCP with every promise removed

The packets look very similar. The handshake is missing and the retries are not as rigorous.

TCP — 20 B header, minimum



UDP — 8 B header, that's all of it



WHO ACTUALLY USES IT

Zoom, WebRTC, game servers, DNS, and every real-time streaming protocol.

A retransmitted audio frame arrives too late to play. Dropping it is the correct behaviour.

THE BROADCAST PROMISE

I liked UDP for broadcast — easy system discovery on a LAN, no registry, no config.

In practice people just hard-coded the IPs they had already allocated. Some popular systems did use it. It never became what I expected.

HOMEWORK

Draw the UDP packet from memory, then capture a real one and diff it against your drawing.

Then answer: with only length and checksum, what does an application have to rebuild itself?

QUIC · HTTP/3

QUIC rebuilds TCP inside UDP, in userspace

HTTP/1.1 · HTTP/2 over TCP

HTTP/2	streams, multiplexed
TLS	separate handshake, separate round trips
TCP	one ordered byte stream — one lost packet stalls every stream
IP	

HTTP/3 over QUIC

HTTP/3	same streams, mapped onto QUIC's
QUIC	independent streams + TLS 1.3, one handshake
UDP	8 bytes the kernel and the middleboxes leave alone
IP	

HEAD-OF-LINE BLOCKING, GONE

HTTP/2 multiplexes streams over one TCP connection — but TCP still delivers one ordered byte stream, so a single lost packet stalls all of them.

QUIC gives each stream its own sequence space. Lose a packet, only that stream waits.

WHY USERSPACE MATTERS

TCP lives in the kernel. Changing it means shipping an OS update to the entire planet and hoping no middlebox objects.

QUIC ships in the browser. Google can deploy a congestion-control change on Tuesday.

WHERE IT ACTUALLY IS

People are still adopting HTTP/2. HTTP/3 is not yet fully QUIC-everywhere.

My read: media over QUIC becomes widespread in the next two to five years.

05

Speak it by hand

Last time we built binary protocols. These three are text — so you can be the client.

SMTP

The S in SMTP is Simple — it has never stood for Secure

A mail transfer is a conversation, and both halves of it are human-readable ASCII.

25

SMTP relay

server to server, plaintext

587

Submission

client to server, STARTTLS — use this one

465

SMTPS

implicit TLS from the first byte

110 / 143

POP3 / IMAP

retrieval, coming up next

WHY IT STAYED TEXT

Every intermediate relay must be able to read, rewrite and add headers. A text protocol with a fixed delimiter is trivially extensible — you add a header nobody understands and nothing breaks.

That is also why forty years of extensions bolted on cleanly: STARTTLS, SPF, DKIM, DMARC are all just headers and verbs.

AND WHY IT STAYED INSECURE

MAIL FROM is whatever you type. There is no authentication in the base protocol at all — you will see this yourself in three slides when you send mail as anyone you like.

SPF, DKIM and DMARC are all later patches trying to answer a question SMTP never asked: is this sender allowed to say that?

Open a real message in Gmail → "Show original" and read the headers before writing any yourself.

Images travel as text, and it costs you 33% to do it

A text protocol cannot carry arbitrary bytes. So the bytes stop being arbitrary.

3 BYTES IN → 4 CHARACTERS OUT



24 bits regrouped as 6 — and 6 bits maps onto 64 safe printable characters.

THE 64 CHARACTERS

A–Z a–z 0–9 + / and = for padding.

Chosen because every mail system, every gateway and every 7-bit link ever built passes them through unchanged.

4/3

size on the wire

≈ 33% overhead, always

76

characters per line

RFC 2045 line-length limit

AND HOW THE PARTS ARE SEPARATED

```
Content-Type: multipart/mixed; boundary="=_a7f3c91"

--=_a7f3c91
Content-Type: text/plain
…the message…

--=_a7f3c91
Content-Type: image/png
Content-Transfer-Encoding: base64
iVBORw0KGgoAAAANSUhEUg…

--=_a7f3c91--
```

Same delimitation idea we used for binary protocols last session — pick a boundary that cannot appear in the body.

One container gives you a whole mail server to talk to

mailpit — SMTP + web UI

```
$ podman run -d \
    --name mailpit \
    -p 1025:1025 \
    -p 8025:8025 \
    axllent/mailpit

# SMTP on 1025, web UI on 8025
# Check mail at: http://localhost:8025
```

MAILPIT

Accepts anything, delivers nothing, and shows you every message in a browser.

Best for watching what you just sent.

greenmail — SMTP + POP3

```
$ podman run -d \
    --name greenmail \
    -e 'GREENMAIL_OPTS=-Dgreenmail.setup.test.all
        -Dgreenmail.hostname=0.0.0.0
        -Dgreenmail.users=bob:secret@localhost
        -Dgreenmail.verbose' \
    -p 3025:3025 \
    -p 3110:3110 \
    docker.io/greenmail/standalone:2.1.12

# SMTP on 3025, POP3 on 3110, user bob / secret
```

GREENMAIL

A real mailbox with a real user, and POP3 on 3110 so we can fetch the message back in the next demo.

This is the one we type into.

Docker works identically — swap podman for docker. Neither container touches the internet; nothing you send here goes anywhere.

DEMO · SMTP

Now be the mail client

you are the client

```
$ telnet localhost 3025

EHLO scaler-demo
MAIL FROM:<jeet@scaler.com>
RCPT TO:<kshitij@scaler.com>
DATA
From: Jeet <jeet@scaler.com>
To: Kshitij <kshitij@scaler.com>
Subject: SMTP is surprisingly simple

Hello K,

We just sent this email by manually speaking SMTP.

.
QUIT
```

THE ENVELOPE VS THE LETTER

MAIL FROM and RCPT TO are the envelope — they decide where it goes.

From: and To: inside DATA are just text printed on the letter. Your mail client shows you the letter.

They do not have to match. Nothing checks.

THE LONE DOT

A single “.” on its own line ends the message. That is the delimiter — and the reason a line in your body starting with a dot gets an extra one added.

Blank line between headers and body. Miss it and your subject becomes body text.

POP is four verbs, and you can hold all of it in your head

IMAP is more popular. POP is easier to understand — so learn this one first.

you are the mail client

```
$ telnet localhost 3110

USER bob
PASS secret
STAT
LIST
RETR 1
DELE 1
QUIT
```

THE WHOLE PROTOCOL

STAT	how many messages, how many bytes
LIST	one line per message: number and size
RETR n	send me message n, in full
DELE n	mark n for deletion — applied at QUIT

PLAINTEXT PASSWORD

USER and PASS go over the wire in the clear. POP3S on 995 exists for exactly this reason.

Send the message on the previous slide to bob@localhost via port 3025, then fetch it back here on 3110. Same bytes, two protocols.

IMAP fixes a lot of problems with POP — you only feel them once you have used POP

POP3 · port 110	IMAP · port 143
Where the mail lives On your device. The server is a spool to be emptied.	**Where the mail lives** On the server. The client holds a cache.
Second device Sees nothing. The first client already took it.	**Second device** Same mailbox, same state, same moment.
Read / unread Local only. Nobody else knows what you have opened.	**Read / unread** A server-side flag. \Seen, \Answered, \Flagged.
Folders There is one. It is called the inbox.	**Folders** Arbitrary hierarchy, created and renamed over the wire.
Search Download everything first, then search locally.	**Search** SEARCH runs on the server. IDLE pushes new mail to you.
Partial fetch No. RETR gives you the whole message, attachments and all.	**Partial fetch** FETCH BODY[1] — one MIME part, headers only, whatever you want.

The trade: POP puts the state on your machine, IMAP puts it on the server. Everything else follows from that one decision.

FRAMING

Length or delimiter. Every protocol today picked one, and some picked both.

DELIMITER

Agree on a byte sequence that ends the message.

```
SMTP       a lone "." on its own line
HTTP/1.1 headers    CRLF CRLF
MIME       --=_boundary--
Redis      CRLF between parts
```

GOOD Cheap to write, human-readable, streams without knowing the size in advance.

COST You must escape the delimiter if it appears in the payload — dot-stuffing. Get that wrong and content becomes protocol.

LENGTH

Put the size in front and read exactly that many bytes.

```
SMS TPDU    TP-UDL = 12 septets
HTTP        Content-Length: 4096
SS7 MTP2    LI, and SIF ≤ 273
Protobuf    varint length prefix
```

GOOD Content can never be confused for protocol. No escaping, no scanning.

COST You must know the size before you start sending — which is exactly what chunked transfer encoding exists to avoid.

BOTH

Delimit the frame, then length-prefix the payload inside it.

```
IMAP        literals: {310} then 310 bytes
HTTP/1.1    CRLF headers + Content-Length
HTTP/2      binary framing over a stream
POP3        +OK line, then a dotted body
```

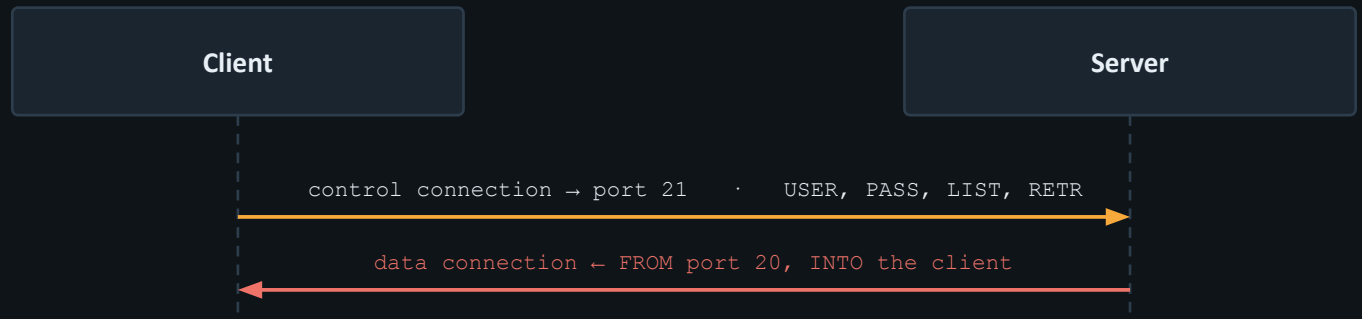
GOOD Human-readable control, safe binary payload. The pragmatic answer.

COST Two parsers, two failure modes, and request smuggling lives in the gap between them.

FTP refuses to mix commands and data at all

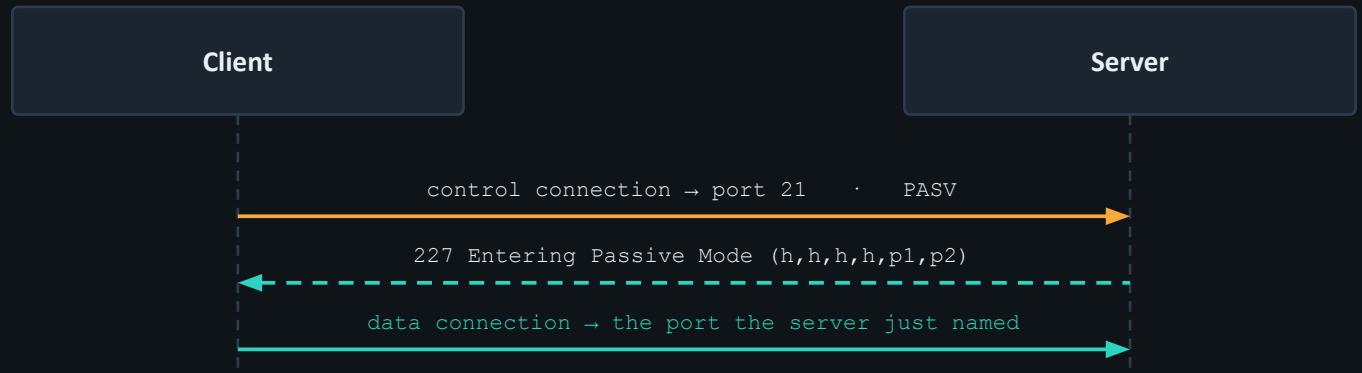
One connection for the conversation. A second, brand-new connection for every file.

ACTIVE MODE — the original design, 1971



The server dials back into you. Every NAT and every firewall on earth now blocks this.

PASSIVE MODE — added later, now effectively mandatory



Both connections are always opened by the client in passive mode — which is the only shape NAT was ever going to allow.

A throwaway FTP server, and a directory to put files in

pyftpdlib

```
$ mkdir -p ftp-demo && echo "hello from ftp" > ftp-demo/hello.txt

$ podman run --rm -it \
    --name ftp-demo \
    -p 2121:2121 \
    -p 30000-30009:30000-30009 \
    -v "$PWD/ftp-demo:/srv" \
    docker.io/library/python:3.13-slim \
    sh -lc 'pip install -q pyftpdlib &&
      python -m pyftpdlib \
        --interface=0.0.0.0 \
        --port=2121 \
        --directory=/srv \
        --username=scaler \
        --password=demo \
        --write \
        --range=30000-30009 \
        --nat-address=127.0.0.1 \
        --debug'
```

--RANGE 30000-30009

The passive-mode data ports, pinned to a small range so we can publish them.

A real server does the same thing so the firewall rule stays finite.

--NAT-ADDRESS

The address the server should advertise in its 227 reply. Inside a container it does not know its own reachable address — so you tell it.

--DEBUG

Leave this on. The server log is the other half of the conversation you are about to have.

DEMO · FTP

Watch a file arrive on a socket you opened yourself

terminal 1 — control

```
$ telnet localhost 2121          # control connection

USER scaler
PASS demo
PWD
LIST
PASV
```

terminal 2 — data

```
$ telnet localhost 30000         # data connection

# nothing yet — the socket is just open

# back in terminal 1:
LIST                             # listing appears HERE

PASV                             # a new port for every transfer
$ telnet localhost 30001
RETR hello.txt                   # file body appears HERE
```

DECODE THE 227 YOURSELF

(127,0,0,1, 117, 48)

The first four octets are the IP address. The last two are the port, as a big-endian pair:

port = 117 × 256 + 48 = 30000

WHY A PORT IS SPLIT ACROSS TWO NUMBERS

Because the reply is text, and in 1971 nobody wanted a client to parse a number larger than 255 out of a comma-separated line.

So a 16-bit port ships as two 8-bit halves, high byte first — network byte order, spelled out in ASCII. The same htons() from the very first slide, written by hand.

HOMEWORK

Four things to go and find out for yourself

01

Draw the UDP packet from memory

Then capture a real one and diff it against your drawing. Note what an application has to rebuild for itself with only length and checksum.

02

TCP SYN cookies are 1996. HTTP cookies are…?

Find out which came first, and whether either name influenced the other. Then list three other places you have pushed state into a token the client hands back.

03

Unpack “Hello World!” by hand

C8 32 9B FD 06 5D DF 72 36 39 04 — eleven octets, twelve characters. Recover the first three by hand, then write the unpacker.

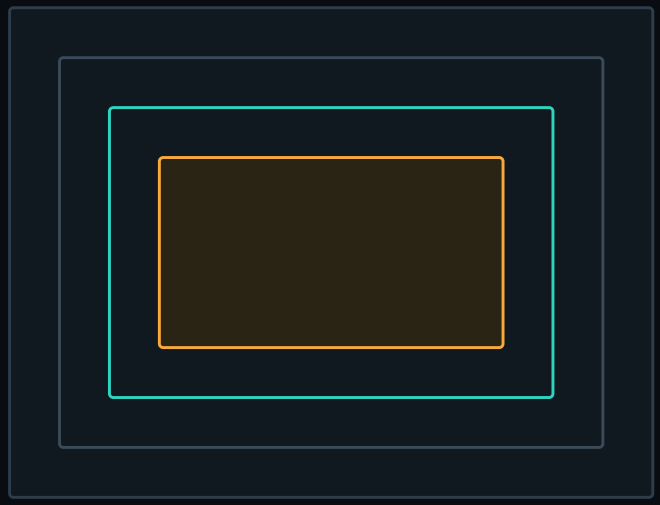
04

Run Asterisk, place a SIP call, capture it

Two softphones and Wireshark. Telephony → VoIP Calls will play the audio straight out of your own capture. Draw your own conclusions about RTP.

WHAT TO KEEP

One layer’s headers are another layer’s body.



Encapsulation

Ethernet ⊃ IP ⊃ TCP ⊃ HTTP is the same picture as MTP2 ⊃ MTP3 ⊃ SCCP ⊃ TCAP ⊃ MAP ⊃ TPDU. Two committees, fifty years apart, one idea.

Framing

Length or delimiter. SMS chose length, SMTP chose a dot, IMAP chose both. There is no third option.

Reliability has an address

SS7 put it at L2, per hop. TCP put it at L4, end to end. QUIC put it in userspace. All three were right for their network.

Control and data want separating

Blue boxes died when SS7 split them. SIP keeps media off the proxy. FTP opens a second socket. Same instinct, three eras.